Assignment_week_5_and_6

Name: R Yadunath Reddy

Roll: 2023BCS0077

Dataset-1 (study_grade)

Steps:

1. Load the necessary libraries

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

2. Read the dataset and split into dependent and independent variables

```
dataset = pd.read_csv('Datasets/dataset1.csv')
hours = np.array(dataset['hours'])
marks = np.array(dataset['marks'])
print(hours)
print(marks)
```

3. Using Gradient Descent with 100 iterations to calculate slope and intercept

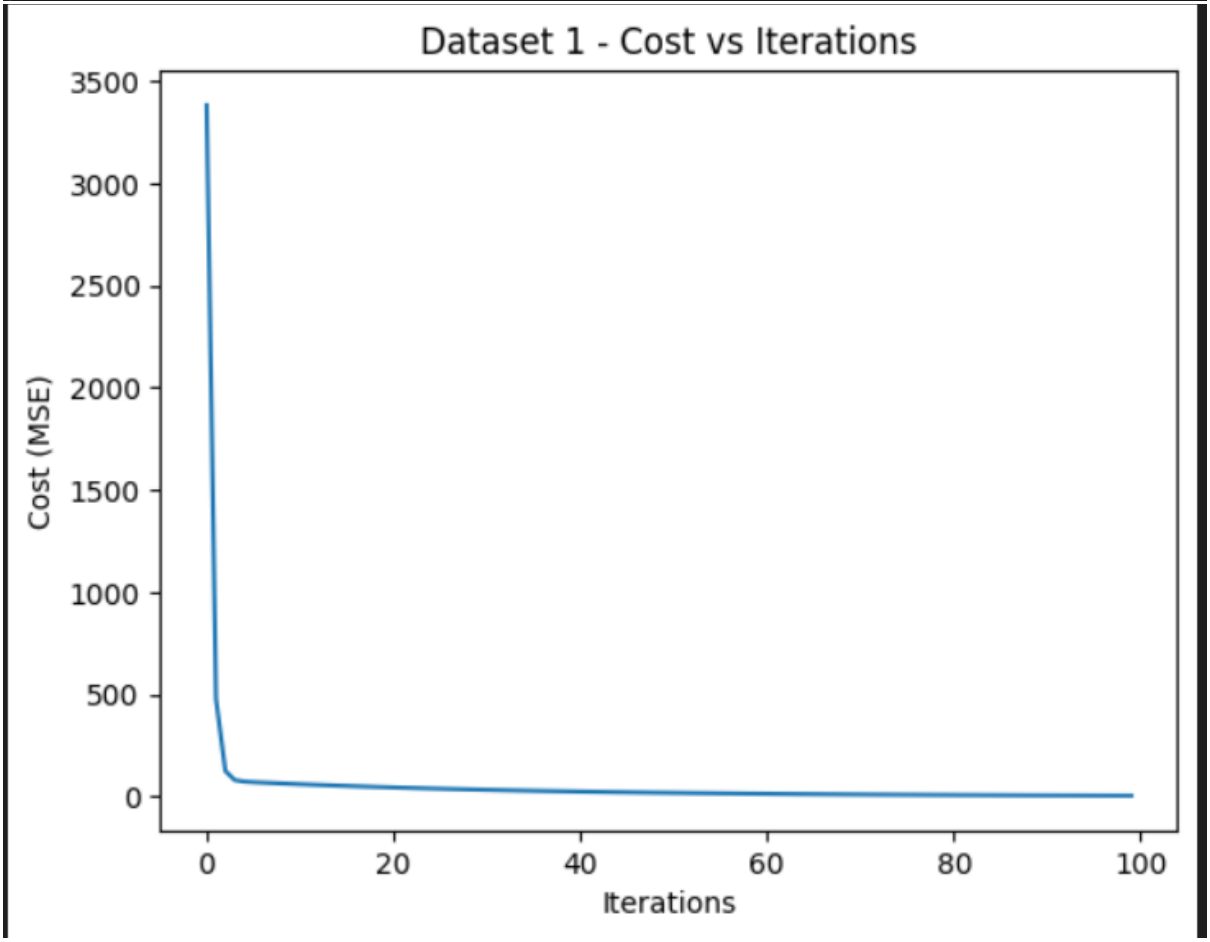
```
m = 0
c = 0
learning_rate = 0.05
iterations = 100
n = len(hours)
X = hours
Y = marks
cost_history = []

for i in range(iterations):
    Y_pred = m*X + c
    cost = (1/n)*np.sum((Y - Y_pred)**2)
    cost_history.append(cost)

    dm = (-2/n)*np.sum(X*(Y - Y_pred))
    dc = (-2/n)*np.sum(Y - Y_pred)

    m = m - learning_rate*dm
    c = c - learning_rate*dc
```

```python
print("Gradient Descent m:", m)
print("Gradient Descent c:", c)
```

```
Gradient Descent m: 10.57668301282294
Gradient Descent c: 21.49530133180529
```

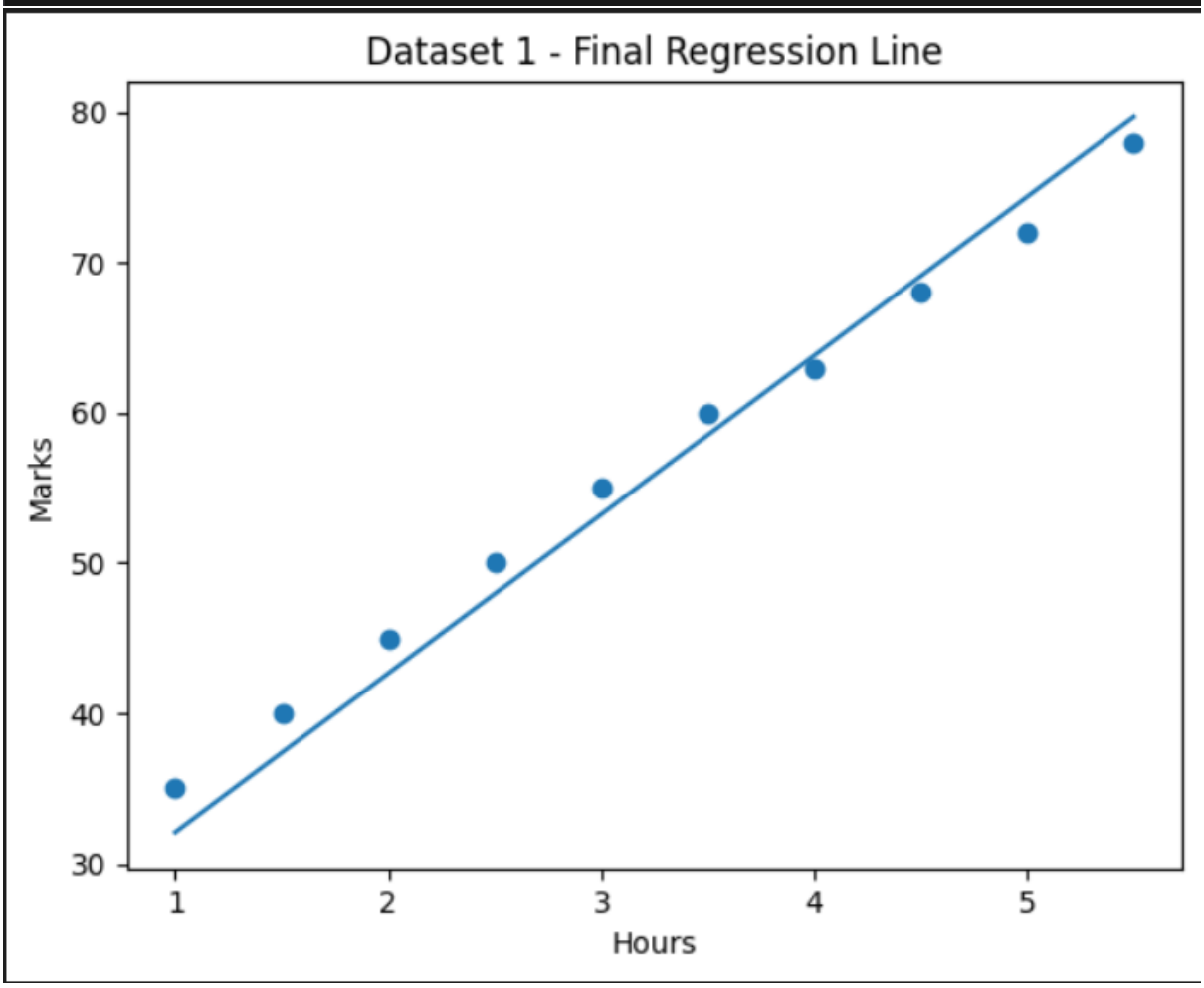
4. Plot the cost vs iterations

```python
plt.figure()
plt.plot(range(iterations), cost_history)
plt.xlabel("Iterations")
plt.ylabel("Cost (MSE)")
plt.title("Dataset 1 - Cost vs Iterations")
plt.show()
```



5. Plot the regression line

```
plt.figure()
plt.scatter(X, Y)
plt.plot(X, m*X + c)
plt.xlabel("Hours")
plt.ylabel("Marks")
plt.title("Dataset 1 - Final Regression Line")
plt.show()
```



6. Predicting for a new value

```
(variable) predicted_marks: int | Any
predicted_marks = m*new_hours + c
print("Prediction for 6 hours:", predicted_marks)
```

```
Prediction for 6 hours: 84.95539940874293
```

Comparison of Parameters (Closed Form vs Gradient Descent)

Closed-form solution gives:

- m = 9.3333
- c = 26.2667

Gradient descent parameters vary significantly with learning rate.

None of the tested learning rates (0.05, 0.01, 0.005) converged to the closed-form values.

For example:

- $\alpha = 0.05 \rightarrow m = 10.57$ (overshoot)
- $\alpha = 0.01 \rightarrow m = 13.61$ (far from optimal)
- $\alpha = 0.005 \rightarrow m = 14.32$ (even farther)

This indicates gradient descent did **not fully converge** within the chosen number of iterations.

Closed-form gives exact optimal parameters instantly.
Gradient descent approximates them and depends heavily on hyperparameters.

Convergence Behaviour of Gradient Descent

The parameters change depending on learning rate.

Since none of the values are close to 9.33 and 26.27, convergence was incomplete.

Likely reasons:

- Insufficient iterations.
- Learning rate not ideal.

Smaller learning rate (0.005) shows slower movement but still not converged.

Larger learning rate (0.05) causes unstable parameter shift.

Conclusion: Gradient descent requires careful tuning to reach the optimal solution.

Effect of Learning Rate

$\alpha = 0.05$ (Relatively Large)

- Parameters overshoot.
- Slope becomes larger than optimal.
- Indicates unstable updates.
- Risk of divergence if increased further.

α = 0.01 (Moderate)

- More stable than 0.05.
- Still far from closed-form values.
- May require more iterations.

α = 0.005 (Smaller)

- Updates are slower.
- Parameters move gradually.
- Would need more iterations for convergence.

What Happens When Learning Rate is Too Small?

- Convergence becomes very slow.
- Parameters change minimally per iteration.
- Requires large number of iterations.

What Happens When Learning Rate is Too Large?

- Parameters overshoot optimal values.
- Cost may oscillate.
- May fail to converge.

Dataset-2 (house_price)

Steps:

1. Load the necessary libraries

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

2. Read the dataset and split into dependent and independent variables

```
dataset = pd.read_csv('Datasets/dataset2.csv')
size = np.array(dataset['house_size_1000_sqft'])
price = np.array(dataset['price_lakhs'])
✓ 0.0s
```

3. Using Gradient Descent with 100 iterations to calculate slope and intercept

```
m = 0
c = 0
learning_rate = 0.05
iterations = 100
n = len(size)
X = size
Y = price

cost_history = []

for i in range(iterations):
    Y_pred = m*X + c
    cost = (1/n)*np.sum((Y - Y_pred)**2)
    cost_history.append(cost)

    dm = (-2/n)*np.sum(X*(Y - Y_pred))
    dc = (-2/n)*np.sum(Y - Y_pred)

    m = m - learning_rate*dm
    c = c - learning_rate*dc
```

✓ 0.0s

```
print("Gradient Descent m:", m)
print("Gradient Descent c:", c)
```

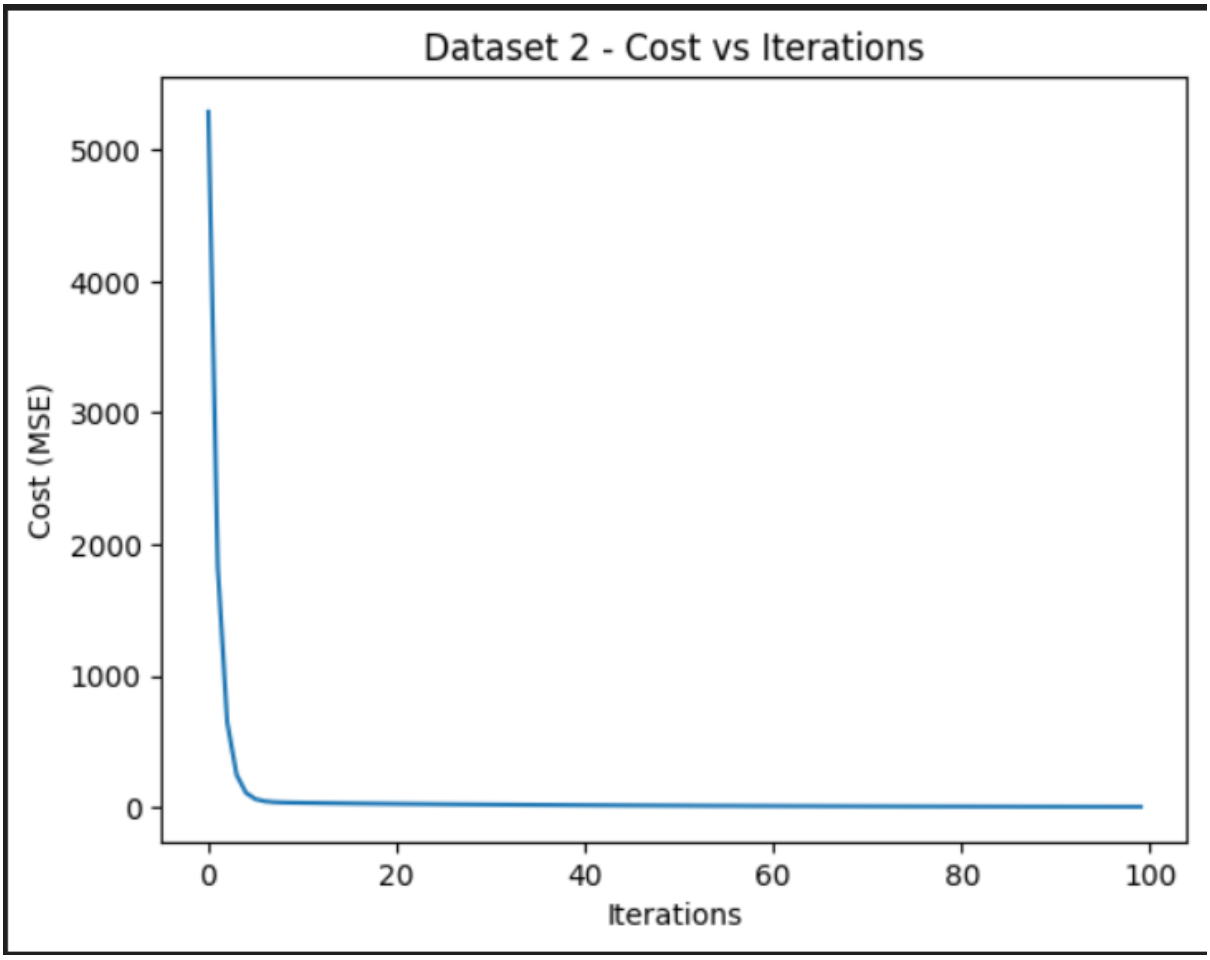
✓ 0.0s

```
Gradient Descent m: 37.56203314369189
Gradient Descent c: 4.708558477824866
```

4. Plot the cost vs iterations

```
plt.figure()
plt.plot(range(iterations), cost_history)
plt.xlabel("Iterations")
plt.ylabel("Cost (MSE)")
plt.title("Dataset 2 - Cost vs Iterations")
plt.show()
```

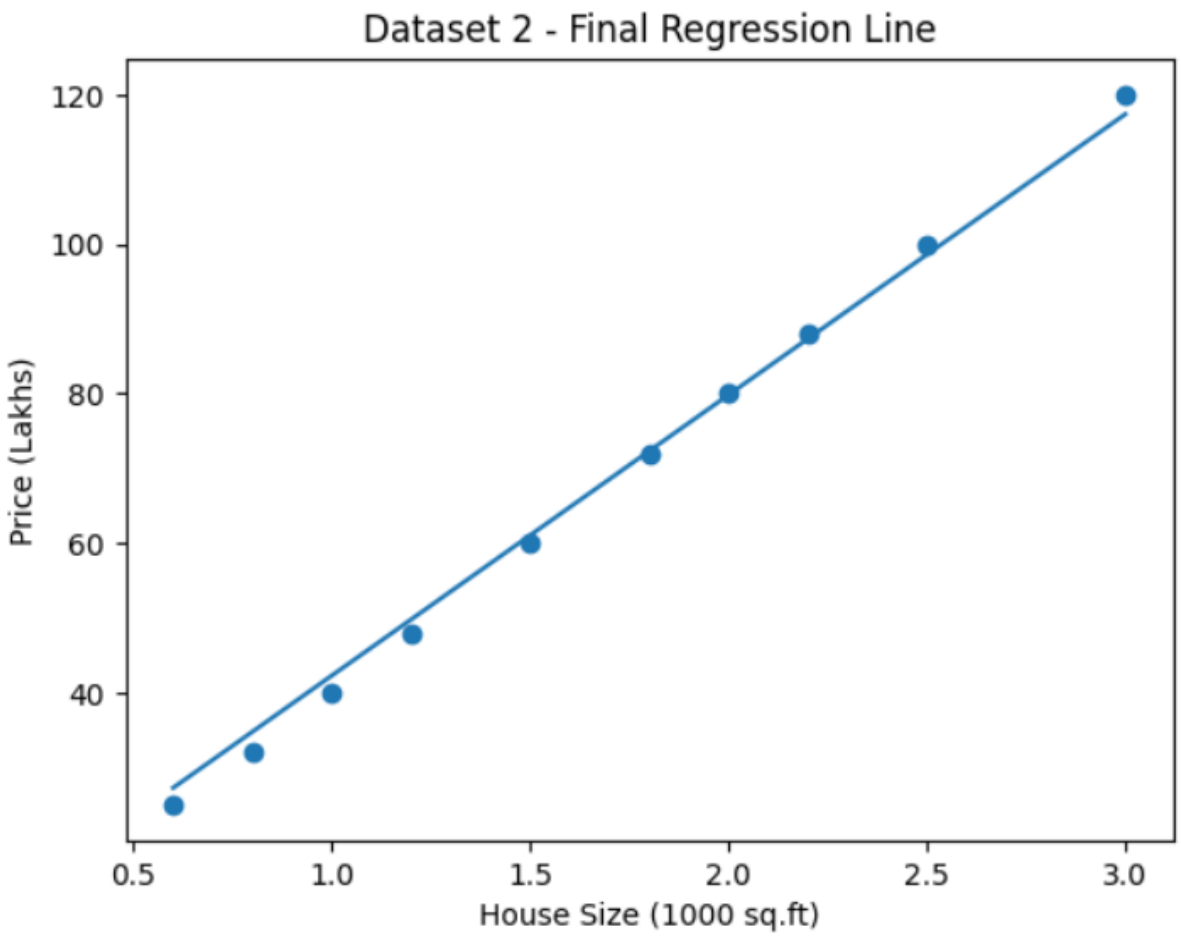
✓ 0.1s



5. Plot the regression line

```
plt.figure()
plt.scatter(X, Y)
plt.plot(X, m*X + c)
plt.xlabel("House Size (1000 sq.ft)")
plt.ylabel("Price (Lakhs)")
plt.title("Dataset 2 - Final Regression Line")
plt.show()
```

✓ 0.1s



6. Predicting for a new value

```
new_size = 2.7
predicted_price = m*new_size + c
print("Prediction for 2.7 (1000 sq.ft):", predicted_price)
```

✓ 0.0s

```
Prediction for 2.7 (1000 sq.ft): 106.12604796579298
```

Comparison of Parameters (Closed Form vs Gradient Descent)

Closed-form solution:

- m = 39.8060
- c = 0.4220

Observations:

- None of the gradient descent results exactly match the closed-form solution.
- α = 0.05 gives closest slope to 39.806.
- Smaller learning rates move more slowly toward optimum.
- Intercept deviates significantly when learning rate is small.

Closed-form again provides exact optimal solution.
Gradient descent approximates but did not fully converge.

Convergence Behaviour

- Dataset 2 shows more sensitivity to learning rate.
- Larger α (0.05) gets closer to optimal slope.
- Smaller α results in slower movement.
- Suggests that dataset scale influences learning rate sensitivity.
-

Effect of Learning Rate

α = 0.05

- Faster updates.
- Closer to optimal solution.
- Risk of instability if increased further.

α = 0.01

- Slower convergence.
- Parameters still far from closed-form.

α = 0.005

- Very slow updates.
- Would need many more iterations.

Too Small Learning Rate

- Very slow convergence.
- Requires more iterations.
- Computationally inefficient.

Too Large Learning Rate

- Parameters may oscillate.
- May overshoot.
- Can diverge.

Observations for different values of Learning Rate

	Dataset-1 (study_grade)		Dataset-2 (house_price)	
Learning Rate	Slope(m)	Intercept(c)	Slope(m)	Intercept(c)
$\alpha = 0.05$	10.57668301282294	21.49530133180529	37.56203314369189	4.708558477824866
$\alpha = 0.01$	13.613324107371728	9.84216441307857	33.34774871594696	12.746254736715787
$\alpha = 0.005$	14.323144654247134	7.118189557025889	32.004340197884495	14.245807286074404